Name: Gargi Devendra Shintre
Div: D15C **Roll No.** 44

Experiment No. 7

Aim:

To build an Artificial Neural Network (ANN) using Keras/TensorFlow.

Theory:

Dataset Link: https://www.kaggle.com/datasets/camnugent/california-housing-prices

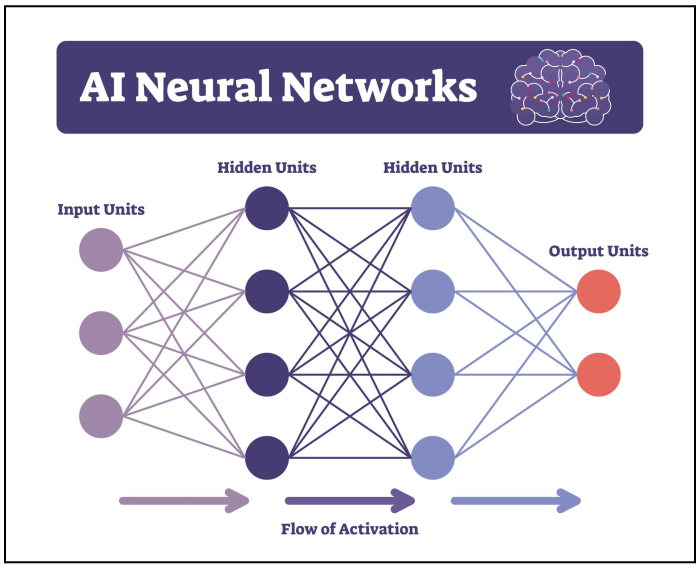
Dataset Description:

This dataset is based on data from the 1990 California census. It serves as an excellent introduction to implementing machine learning algorithms for regression tasks.

- **Objective:** To predict the **median house value** for California districts based on various features.
- **Records:** 20,640 instances.
- **Target:** median_house_value (Continuous Numerical).
- **Features:** 9 predictors including location (lat/long), age, rooms, population, income, and proximity to the ocean.

Artificial Neural Network (ANN)

An Artificial Neural Network (ANN) is a computational model inspired by the human brain's biological neural networks. It consists of interconnected groups of artificial neurons that process information using a connectionist approach to computation.



Working Principle:

- Input Layer: Receives the raw feature data (e.g., median income, rooms).
- Hidden Layers: Perform mathematical transformations. Each neuron computes a weighted sum of inputs $(z = \sum w_i x_i + b)$
- and passes it through a Non-Linear Activation Function (like ReLU). This allows the network to learn complex patterns.
- Output Layer: Produces the final prediction. For regression, this is usually a single neuron with a Linear activation function.
- Backpropagation: The network compares the prediction to the actual value (Loss), calculates the

error, and updates the weights backwards to minimize this error using an optimizer (like Adam or SGD).

Workflow:

1. **Data Preprocessing:**
    - **Imputation:** The column total_bedrooms has missing values. We fill these with the median value.
    - **Encoding:** Convert the categorical text column ocean_proximity into numeric format using One-Hot Encoding.
    - **Feature Scaling:** Neural Networks converge much faster and accurately if data is scaled. We use StandardScaler to normalize inputs (mean=0, std=1).
2. **Splitting:** Partition data into Train (80%) and Test (20%) sets.
3. **Model Architecture:**
    - **Input Layer:** Matches the number of features.
    - **Hidden Layers:** Two Dense layers with ReLU activation (e.g., 64 and 32 neurons).
    - **Output Layer:** One Dense layer with Linear activation (for regression output).
4. **Compilation:**
    - **Optimizer:** Adam (Adaptive Moment Estimation).
    - **Loss Function:** Mean Squared Error (MSE).
5. **Training:** Train for 50-100 epochs with a validation split to monitor overfitting.
6. **Evaluation:** Calculate R2 Score and RMSE on test data.

Performance Analysis

1. Training Performance

- **Loss Curve:** By plotting Training Loss vs. Validation Loss, we observe that both decrease rapidly in the first 10 epochs and then stabilize. If Validation Loss starts increasing while Training Loss decreases, it indicates overfitting.
- **Convergence:** The model converges efficiently due to the Adam optimizer and scaled data.

2. Prediction Accuracy

- **R-squared Score:** Typically achieves 0.75 - 0.85 on this dataset with a basic ANN.
- **RMSE:** The Root Mean Squared Error gives us the average deviation of our price prediction in dollars.
- **Comparison:** ANN generally outperforms simple Linear Regression on this dataset because it captures the non-linear relationship between location (lat/long) and price, which simple linear models miss.

Hyperparameter Tuning:

Hyperparameter	Description	Effect
Number of Layers	Depth of network	More layers allow learning more complex features (e.g., interaction between income and location).
Neurons per Layer	Width of network	More neurons increase capacity but risk

		overfitting. We used 64 and 32.
Activation Function	ReLU vs. Tanh vs. Sigmoid	ReLU is standard for hidden layers as it prevents the vanishing gradient problem.
Batch Size	Samples per update	Smaller batches (e.g., 32) offer more robust updates; larger batches (e.g., 512) are faster.
Epochs	Number of passes	Too few = Underfitting; Too many = Overfitting.

Algorithm Limitations:

1. **"Black Box" Nature:** Unlike Decision Trees or Linear Regression, it is very difficult to interpret *why* the ANN made a specific price prediction. We cannot easily see which feature (e.g., income vs. rooms) was most important.
2. **Data Hungry:** ANNs typically require large amounts of data to outperform classical ML algorithms. On small datasets, they may overfit.
3. **Computational Cost:** Training takes significantly longer than standard regression models and requires more hardware resources (GPU recommended for very large nets).
4. **Sensitive to Scaling:** If one feature ranges from 0-1 and another from 0-10000, the network will fail to learn properly without scaling.

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.model_selection import
train_test_split
from sklearn.preprocessing import
StandardScaler
from sklearn.metrics import r2_score,
mean_squared_error, mean_absolute_error

#
═══════════════════════════════════════
═══
# 1. LOAD AND PREPROCESS DATA
#
═══════════════════════════════════════
═══

# 1. Load Dataset
try:
    df = pd.read_csv('housing.csv')
    print("Dataset loaded
successfully.")
except:
    print("Error: Please upload
'housing.csv' to Colab.")

# 2. Handling Missing Values
# 'total_bedrooms' has missing values.
Fill with median.
df['total_bedrooms'] =
df['total_bedrooms'].fillna(df['total_b
edrooms'].median())

# 3. Encoding Categorical Data
# 'ocean_proximity' is text. Convert to
One-Hot Encoding.
df = pd.get_dummies(df,
columns=['ocean_proximity'],
drop_first=True)

# 4. Feature Selection
```

```
X = df.drop('median_house_value',
axis=1)
y = df['median_house_value']

# 5. Split Data (80% Train, 20% Test)
X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.2,
random_state=42)

# 6. Feature Scaling (CRITICAL for ANN)
scaler = StandardScaler()
X_train_scaled =
scaler.fit_transform(X_train)
X_test_scaled =
scaler.transform(X_test)

print(f"Training Shape:
{X_train_scaled.shape}")
print(f"Testing Shape:
{X_test_scaled.shape}")

#
═══════════════════════════════════════
═══
# 2. BUILDING THE ANN MODEL
#
═══════════════════════════════════════
═══
print("\n" + "="*50)
print("BUILDING NEURAL NETWORK")
print("="*50)

model = keras.Sequential([
    # Input Layer (implicitly defined
by input_shape) + 1st Hidden Layer
    layers.Dense(64, activation='relu',
input_shape=[X_train_scaled.shape[1]]),

    # 2nd Hidden Layer
    layers.Dense(32,
activation='relu'),

    # Output Layer (1 neuron, linear
activation for Regression)
    layers.Dense(1,
activation='linear')
])

# Compile the model
model.compile(
    optimizer='adam',
    loss='mean_squared_error',
    metrics=['mean_absolute_error']
)

model.summary()

#
═══════════════════════════════════════
═══
# 3. TRAINING THE MODEL
#
═══════════════════════════════════════
═══
print("\n" + "="*50)
print("TRAINING…")
print("="*50)

# Train for 100 epochs, keep 20% of
training data for validation
history = model.fit(
    X_train_scaled, y_train,
    validation_split=0.2,
    batch_size=32,
    epochs=100,
    verbose=0 # Set to 1 to see
per-epoch logs
)

print("Training Complete.")

# Plot Training History (Loss Curves)
hist_df = pd.DataFrame(history.history)
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(hist_df['loss'], label='Train
Loss')
plt.plot(hist_df['val_loss'],
label='Val Loss')
plt.title('Model Loss (MSE)')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(hist_df['mean_absolute_error']
, label='Train MAE')
plt.plot(hist_df['val_mean_absolute_err
or'], label='Val MAE')
plt.title('Mean Absolute Error')
plt.xlabel('Epochs')
plt.ylabel('MAE')
plt.legend()
plt.grid(True)
plt.show()
```

```
#
═══════════════════════════════════════
═══
# 4. EVALUATION
#
═══════════════════════════════════════
═══
print("\n" + "="*50)
print("EVALUATION ON TEST SET")
print("="*50)

y_pred =
model.predict(X_test_scaled).flatten()

# Metrics
r2 = r2_score(y_test, y_pred)
rmse =
np.sqrt(mean_squared_error(y_test,
y_pred))
mae = mean_absolute_error(y_test,
y_pred)

print(f"R-squared Score: {r2:.4f}
({r2*100:.2f}%)")
print(f"Root Mean Squared Error (RMSE):
${rmse:.2f}")
print(f"Mean Absolute Error (MAE):
${mae:.2f}")

# Visualizing Predictions vs Actual
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, alpha=0.3,
color='blue')
plt.plot([y.min(), y.max()], [y.min(),
y.max()], 'r--', lw=2)
plt.xlabel('Actual Price')
plt.ylabel('Predicted Price')
plt.title('Actual vs Predicted Housing
Prices (ANN)')
plt.grid(True)
plt.show()
```

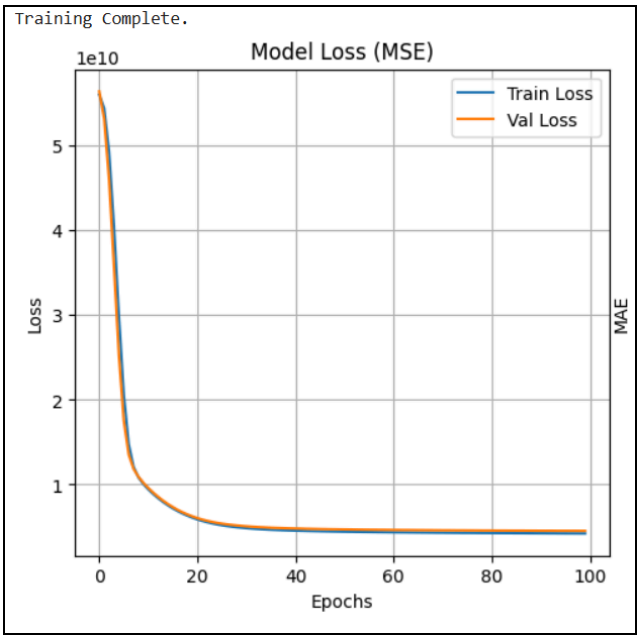
Output:

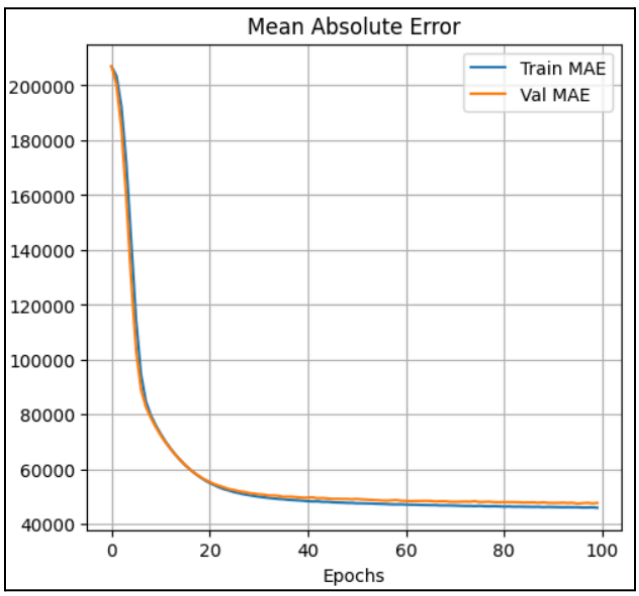
Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	832
dense_1 (Dense)	(None, 32)	2,080
dense_2 (Dense)	(None, 1)	33

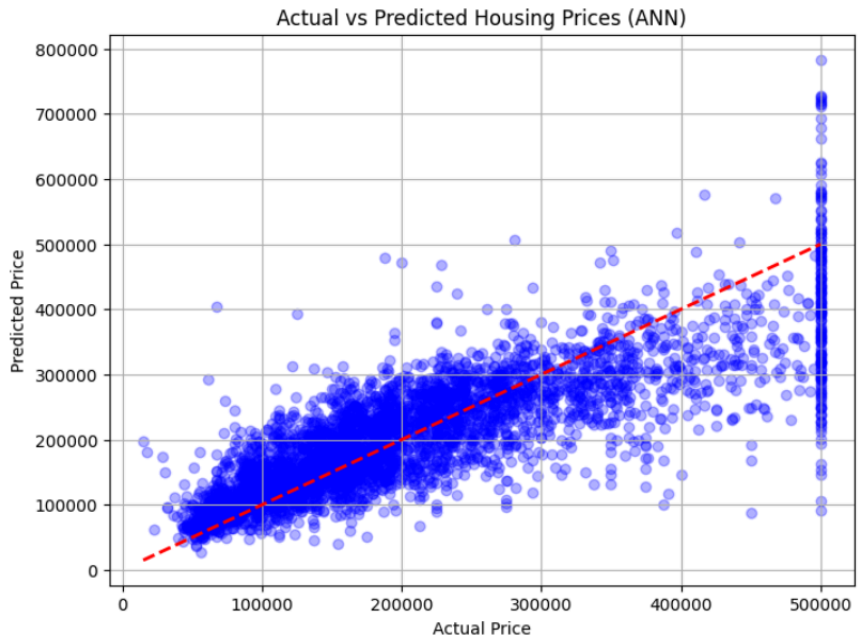
Total params: 2,945 (11.50 KB)
Trainable params: 2,945 (11.50 KB)
Non-trainable params: 0 (0.00 B)

```
Training Complete.
```





```
==================================================
EVALUATION ON TEST SET
==================================================
129/129 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
R-squared Score: 0.6571 (65.71%)
Root Mean Squared Error (RMSE): $67029.66
Mean Absolute Error (MAE): $47780.23
```



Conclusion:

In this experiment, we successfully built a deep learning regression model using Keras/TensorFlow to predict housing prices. The ANN demonstrated strong predictive power, capturing non-linear patterns in the data (particularly geospatial data) that traditional linear models struggle with. The experiment highlighted the critical importance of Data Preprocessing (imputation, scaling) in the Deep Learning pipeline.